

VAO랑 VBO 헷갈리는 사람 많아. 비유부터 시작하고 코드로 연결해줄게.

VBO vs VAO — 제대로 이해하기

비유로 먼저 잡기

VBO는 창고, VAO는 창고 사용 설명서라고 생각해.

창고(VBO)에 박스들이 쌓여 있어. 근데 창고만 있으면 안 돼. "3번 선반 왼쪽에서 4번째 박스, 12바이트씩 건너뛰면서 읽어라"는 설명서(VAO)가 있어야 GPU가 데이터를 제대로 꺼낼 수 있거든.

VBO — 데이터를 GPU 메모리에 올리는 것

VBO는 그냥 GPU 메모리의 버퍼야. 정점 위치, 색상, 법선 같은 데이터를 CPU에서 GPU로 옮겨놓는 공간.

```
GLuint vbo;
glGenBuffers(1, &vbo);           // 버퍼 ID 하나 발급
glBindBuffer(GL_ARRAY_BUFFER, vbo); // 이 버퍼를 활성화
glBufferData(                      // 실제 데이터 GPU에 업로드
    GL_ARRAY_BUFFER,
    sizeof(vertices),             // 데이터 크기 (바이트)
    vertices,                     // CPU 메모리 주소
    GL_STATIC_DRAW                // 자주 안 바꿀 거야 → GPU가 최적 위치에 저장
);
```

VBO 안에는 그냥 바이트 덩어리가 들어있어. GPU 입장에서는 "뭔가 데이터가 있다"는 것만 알고, 이게 위치인지 색 상인지 몇 개인지는 모름.

VAO — "이 데이터를 어떻게 읽어라" 는 설명서

VAO는 데이터 자체가 아니라 데이터를 읽는 방법을 저장하는 객체야. 구체적으로는 이런 정보들을 저장해.

- 어떤 VBO에서 데이터를 읽을 건지
- 데이터가 몇 바이트짜리인지 (vec2? vec3? vec4?)
- 데이터가 몇 바이트 간격으로 있는지 (stride)
- 데이터가 버퍼 어디서부터 시작하는지 (offset)
- 어떤 shader location에 연결할 건지

```
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao); // 지금부터 하는 설정을 이 VAO에 기록

glVertexAttribPointer(
    0,                // shader의 location = 0 에 연결
    3,                // vec3 → 한 번에 float 3개
    GL_FLOAT,         // 타입
```

```

    GL_FALSE, // 정규화 안 함
    0, // stride: 0 = 뽁뽁하게 붙어있음 (자동 계산)
    (void*)0 // offset: 버퍼 시작점부터
);
glEnableVertexAttribArray(0); // location=0 활성화

```

이 설정이 VAO 안에 기록됨. 나중에 `glBindVertexArray(vao)` 하나만 불러도 이 설정이 통째로 복원돼.

위치 + 색상을 따로 VBO 두 개로 넘기는 경우

실제로는 위치 데이터와 색상 데이터를 따로 저장하는 경우가 많아.

```

// --- VBO 두 개 준비 ---
GLuint vbo_pos, vbo_color;

// 위치 데이터 VBO
glGenBuffers(1, &vbo_pos);
glBindBuffer(GL_ARRAY_BUFFER, vbo_pos);
glBufferData(GL_ARRAY_BUFFER, sizeof(positions), positions, GL_STATIC_DRAW);

// 색상 데이터 VBO
glGenBuffers(1, &vbo_color);
glBindBuffer(GL_ARRAY_BUFFER, vbo_color);
glBufferData(GL_ARRAY_BUFFER, sizeof(colors), colors, GL_STATIC_DRAW);

// --- VAO에 두 개의 연결 정보 기록 ---
GLuint vao;
glGenVertexArrays(1, &vao);
glBindVertexArray(vao);

// location=0 → vbo_pos에서 vec3 읽어라
glBindBuffer(GL_ARRAY_BUFFER, vbo_pos);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(0);

// location=1 → vbo_color에서 vec3 읽어라
glBindBuffer(GL_ARRAY_BUFFER, vbo_color);
glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 0, 0);
glEnableVertexAttribArray(1);

```

셰이더에서는 이렇게 받아.

```

layout (location = 0) in vec3 vPosition; // vbo_pos에서 옴
layout (location = 1) in vec3 vColor;    // vbo_color에서 옴

```

VAO 하나가 VBO 여러 개와의 연결 정보를 동시에 저장하고 있는 거야.

드로우콜 시점

```
// 렌더 루프 안에서
glBindVertexArray(vao);           // 이거 하나로 모든 연결 정보 복원
glDrawArrays(GL_TRIANGLES, 0, 3); // 그리기
```

VAO를 바인딩하면 그 안에 기록된 "어떤 VBO에서 어떻게 읽어라"는 설정이 전부 복원돼. VBO를 다시 바인딩할 필요 없음.

한 줄 요약

	VBO	VAO
정체	GPU 메모리 버퍼	읽기 방법 저장 객체
내용	실제 정점 데이터 (바이트)	"어디서 어떻게 읽어라" 설정
비유	창고	창고 사용 설명서
드로우콜 시	직접 안 건드림	애만 바인딩하면 됨

- 🔥 VAO 없이 VBO만 있으면 GPU가 데이터를 꺼낼 수 없음. VBO 없이 VAO만 있으면 참조할 데이터가 없음. 항상 **VBO로 데이터 올리고 → VAO로 읽기 방법 등록** 순서로 써야 함.
- 🔥 물체마다 VAO를 하나씩 만들어두면, 드로우할 때 `glBindVertexArray(해당_vao)` 하나만 불러도 돼서 코드가 깔끔해짐. 물체 10개면 VAO 10개, VBO는 필요에 따라 여러 개.